

**LAPORAN PRAKTIKUM PEKAN 6**



**MATA KULIAH STRUKTUR DATA**

**DOSEN PENGAMPU :**

**Dr. WAHYUDI, S.T., M.T.**

**OLEH :**

**MUHAMMAD HANS NAFIS**

**NIM : 2511532027**

**FAKULTAS TEKNOLOGI INFORMASI**

**DEPARTEMEN INFORMATIKA**

**UNIVERSITAS ANDALAS**

**2026**

## KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 6 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Double Linked List*.

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar struktur data serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 12 Mei 2026

Muhammad Hans Nafis

## DAFTAR ISI

|                                     |    |
|-------------------------------------|----|
| KATA PENGANTAR.....                 | i  |
| DAFTAR ISI.....                     | ii |
| BAB 1 .....                         | 1  |
| PENDAHULUAN.....                    | 1  |
| 1.1 Latar Belakang.....             | 1  |
| 1.2 Tujuan Praktikum .....          | 1  |
| 1.3 Manfaat Praktikum .....         | 1  |
| BAB 2 .....                         | 2  |
| PEMBAHASAN .....                    | 2  |
| 2.1 <i>Double Linked List</i> ..... | 2  |
| 2.2 Class NodeDLL .....             | 3  |
| 2.3 Class InsertDLL .....           | 4  |
| 2.4 Class PenelusuranDLL.....       | 8  |
| 2.5 Class HapusDLL .....            | 10 |
| 2.6 Tugas Praktikum Pekan 6.....    | 15 |
| BAB 3 .....                         | 27 |
| KESIMPULAN.....                     | 27 |
| DAFTAR PUSTAKA .....                | 28 |
| LAMPIRAN.....                       | 29 |

# BAB 1

## PENDAHULUAN

### 1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang digunakan untuk mengatur dan menyimpan data agar dapat diolah secara efisien. Dalam pengembangan program, pemilihan struktur data yang tepat sangat berpengaruh terhadap kemudahan pengelolaan data serta performa program. Salah satu struktur data yang sering digunakan adalah *linked list*, yaitu struktur data linear yang terdiri dari kumpulan node yang saling terhubung menggunakan pointer atau referensi. *Linked list* memiliki beberapa jenis salah satunya adalah *double linked list*.

*Double Linked List* (DLL) merupakan pengembangan dari *single linked list* yang memiliki dua buah pointer pada setiap node, yaitu pointer menuju node berikutnya (*next*) dan pointer menuju node sebelumnya (*prev*). Dengan adanya dua arah hubungan tersebut, proses traversal data dapat dilakukan dari depan ke belakang maupun dari belakang ke depan. Selain itu, operasi penyisipan dan penghapusan data menjadi lebih fleksibel dibandingkan dengan *single linked list* karena node dapat diakses dari kedua arah.

### 1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar struktur data *Double Linked List*.
2. Mengetahui cara kerja node dan pointer dalam *Double Linked List*.
3. Mengetahui dan memahami operasi dasar pada *Double Linked List*.

### 1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai konsep dasar struktur data, khususnya *Double Linked List*.
2. Meningkatkan kemampuan pemrograman dalam bahasa Java.
3. Melatih keterampilan dalam mengelola kode program secara terstruktur.

## BAB 2

### PEMBAHASAN

#### 2.1 *Double Linked List*

*Double Linked List* (DLL) merupakan salah satu jenis struktur data linear yang terdiri dari kumpulan node yang saling terhubung satu sama lain. Setiap node pada *double linked list* memiliki tiga bagian utama, yaitu data, pointer menuju node berikutnya (*next*), dan pointer menuju node sebelumnya (*prev*). Berbeda dengan *single linked list* yang hanya memiliki satu arah hubungan antar node, *double linked list* memungkinkan perpindahan data dilakukan secara dua arah, yaitu maju dan mundur. Hal ini membuat proses traversal data menjadi lebih fleksibel dan efisien dalam beberapa kondisi tertentu.

Pada *double linked list*, node pertama disebut *head* dan node terakhir disebut *tail*. pointer *prev* pada *head* akan bernilai *null* karena tidak memiliki node sebelumnya, sedangkan pointer *next* pada *tail* juga bernilai *null* karena tidak memiliki node setelahnya. hubungan dua arah antar node membuat struktur data ini lebih mudah digunakan dalam operasi tertentu, seperti penghapusan node di tengah list atau perpindahan data dari belakang ke depan tanpa perlu melakukan traversal ulang dari *head*.

Operasi dasar yang terdapat pada *double linked list* meliputi *insertion*, *deletion*, *traversal*, dan *searching*. Proses *insertion* atau penambahan data dapat dilakukan di awal list, di akhir list, maupun di tengah list. Ketika sebuah node baru ditambahkan, pointer *next* dan *prev* harus diperbarui agar hubungan antar node tetap terhubung dengan benar. Jika salah satu pointer tidak diperbarui dengan tepat, maka struktur *linked list* dapat menjadi rusak dan menyebabkan kesalahan saat program dijalankan.

Selain *insertion*, operasi *deletion* atau penghapusan data juga menjadi bagian penting dalam *double linked list*. Pada proses ini, node yang akan dihapus harus dilepaskan dari hubungan node sebelumnya dan node sesudahnya dengan memperbarui pointer *prev* dan *next*. Keunggulan *double linked list* dibanding *single linked list* terlihat pada proses penghapusan node tertentu karena akses terhadap node sebelumnya dapat dilakukan secara langsung melalui pointer *prev* tanpa harus melakukan pencarian ulang dari awal list.

*Traversal* pada *double linked list* dapat dilakukan dalam dua arah, yaitu *forward traversal* dan *backward traversal*. *Forward traversal* dilakukan mulai dari *head* menuju *tail* menggunakan pointer *next*, sedangkan *backward traversal* dilakukan dari *tail* menuju *head* menggunakan pointer *prev*. Kemampuan traversal dua arah ini menjadi salah satu kelebihan utama *double linked list* karena memudahkan proses navigasi data, terutama pada aplikasi yang memerlukan perpindahan maju dan mundur secara cepat.

Dalam implementasi pemrograman, *double linked list* sering digunakan pada berbagai aplikasi komputer, seperti fitur *undo* dan *redo* pada editor teks, navigasi halaman pada browser, playlist musik, serta sistem manajemen data yang membutuhkan akses dua arah. Penggunaan struktur data ini memberikan fleksibilitas lebih tinggi dibandingkan *array* karena ukuran list dapat berubah secara dinamis sesuai kebutuhan tanpa harus menentukan kapasitas data di awal program.

## 2.2 Class NodeDLL

Berikut kode program Java mengenai class NodeDLL :

```
public class NodeDLL_2511532027 {  
    // mendefinisikan kelas node  
    int data_2027; // data  
    NodeDLL_2511532027 next_2027; // Pointer ke next node  
    NodeDLL_2511532027 prev_2027; // Pointer ke prev node  
  
    // konstruktor  
    public NodeDLL_2511532027(int data_2027) {  
        this.data_2027 = data_2027;  
        this.next_2027 = null;  
        this.prev_2027 = null;  
    }  
}
```

Kode program tersebut digunakan untuk mendefinisikan sebuah node pada struktur data *double linked list*. Kelas NodeDLL\_2511532027 berfungsi sebagai representasi dari setiap elemen yang terdapat di dalam *linked list*. Di dalam kelas tersebut terdapat variabel *data\_2027*, *next\_2027*, dan *prev\_2027*. Variabel *data\_2027* bertipe integer yang digunakan untuk menyimpan nilai data pada node. Selain itu, terdapat variabel *next\_2027* yang berfungsi sebagai pointer atau referensi

menuju node berikutnya, serta variabel `prev_2027` yang digunakan untuk menunjuk node sebelumnya. Kedua pointer tersebut merupakan ciri utama dari *double linked list* karena memungkinkan traversal dilakukan dalam dua arah. Jika suatu node belum memiliki hubungan dengan node lain, maka pointer akan bernilai *null*.

Program juga memiliki konstruktor `NodeDLL_2511532027(int data_2027)` yang digunakan untuk menginisialisasi objek node saat pertama kali dibuat. Konstruktor ini menerima parameter berupa data yang akan disimpan ke dalam node melalui perintah `this.data_2027 = data_2027`. Selanjutnya, pointer `next_2027` dan `prev_2027` diatur bernilai *null* karena pada saat node baru dibuat, node tersebut belum terhubung dengan node lain. Konstruktor ini mempermudah proses pembuatan node baru pada operasi penambahan data di dalam *double linked list*.

### 2.3 Class InsertDLL

Berikut kode program Java mengenai class `InsertDLL` :

```
public class InsertDLL_2511532027 {
    // menambahkan node di awal DLL
    static NodeDLL_2511532027 insertBegin_2027(NodeDLL_2511532027
head_2027, int data_2027) {
        // buat node baru
        NodeDLL_2511532027 new_node_2027 = new
NodeDLL_2511532027(data_2027);
        // jadikan pointer nextnya head
        new_node_2027.next_2027 = head_2027;
        // jadikan pointer prev head ke new_node
        if (head_2027 != null) {
            head_2027.prev_2027 = new_node_2027;
        }
        return new_node_2027;
    }
    // fungsi menambahkan node di akhir
    public static NodeDLL_2511532027
insertEnd_2027(NodeDLL_2511532027 head_2027, int newData_2027) {
        // buat node baru
        NodeDLL_2511532027 newNode_2027 = new
NodeDLL_2511532027(newData_2027);
        // jika dll null jadikan head
        if (head_2027 == null) {
```

```

        head_2027 = newNode_2027;
    }
    else {
        NodeDLL_2511532027 curr_2027 = head_2027;
        while (curr_2027.next_2027 != null) {
            curr_2027 = curr_2027.next_2027;
        }
        curr_2027.next_2027 = newNode_2027;
        newNode_2027.prev_2027 = curr_2027;
    }
    return head_2027;
}

// fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511532027
insertAtPosition_2027(NodeDLL_2511532027 head_2027, int pos_2027, int
new_data_2027) {
    // buat node baru
    NodeDLL_2511532027 new_node_2027 = new
NodeDLL_2511532027(new_data_2027);
    if (pos_2027 == 1) {
        new_node_2027.next_2027 = head_2027;
        if (head_2027 != null) {
            head_2027.prev_2027 = new_node_2027; }
        head_2027 = new_node_2027;
        return head_2027; }
    NodeDLL_2511532027 curr_2027 = head_2027;
    for (int i_2027 = 1; i_2027 < pos_2027 - 1 && curr_2027 !=
null; ++i_2027) {
        curr_2027 = curr_2027.next_2027; }
    if (curr_2027 == null) {
        System.out.println("Posisi tidak ada");
        return head_2027; }
    new_node_2027.prev_2027 = curr_2027;
    new_node_2027.next_2027 = curr_2027.next_2027;
    curr_2027.next_2027 = new_node_2027;
    if (new_node_2027.next_2027 != null) {
        new_node_2027.next_2027.prev_2027 = new_node_2027; }
    return head_2027;
}

```



```

    }
    public static void printList_2027(NodeDLL_2511532027 head_2027) {
        NodeDLL_2511532027 curr_2027 = head_2027;
        while (curr_2027 != null) {
            System.out.print(curr_2027.data_2027 + " <-> ");
            curr_2027 = curr_2027.next_2027;
        }
        System.out.println();
    }

    public static void main (String[] args) {
        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511532027 head_2027 = new NodeDLL_2511532027(2);
        head_2027.next_2027 = new NodeDLL_2511532027(3);
        head_2027.next_2027.prev_2027 = head_2027;
        head_2027.next_2027.next_2027 = new NodeDLL_2511532027(5);
        head_2027.next_2027.next_2027.prev_2027 =
head_2027.next_2027;
        // cetak DLL awal
        System.out.print("DLL Awal: ");
        printList_2027(head_2027);
        // tambah 1 di awal
        head_2027 = insertBegin_2027(head_2027, 1);
        System.out.print(
            "simpul 1 ditambahkan di awal: ");
        printList_2027(head_2027);
        // tambah 6 di akhir
        System.out.print(
            "simpul 6 ditambah di akhir: ");
        int data_2027 = 6;
        head_2027 = insertEnd_2027(head_2027, data_2027);
        printList_2027(head_2027);
        // menambahkan node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4 ");
        int data2_2027 = 4;
        int pos_2027 = 4;
        head_2027 = insertAtPosition_2027(head_2027, pos_2027,
data2_2027);
        printList_2027(head_2027);
    }
}

```

```
}  
}
```

Kode program `InsertDLL_2511532027` digunakan untuk melakukan berbagai operasi penambahan node pada struktur data *double linked list*. Program ini memiliki beberapa fungsi utama, yaitu menambahkan node di awal list, di akhir list, dan pada posisi tertentu. Selain itu, program juga menyediakan fungsi untuk menampilkan isi *linked list* serta fungsi main untuk menjalankan dan menguji seluruh operasi yang telah dibuat. Dengan adanya berbagai fungsi tersebut, program mampu menunjukkan cara kerja manipulasi data pada *double linked list* secara lengkap.

Fungsi `insertBegin_2027()` digunakan untuk menambahkan node baru di bagian awal *double linked list*. Pada fungsi ini, program terlebih dahulu membuat node baru menggunakan konstruktor `NodeDLL_2511532027`. Setelah node dibuat, pointer `next_2027` dari node baru diarahkan ke *head* lama sehingga node baru berada di depan list. Jika *linked list* sebelumnya tidak kosong, maka pointer `prev_2027` dari *head* lama diubah agar menunjuk ke node baru. Proses ini menunjukkan bahwa penambahan data di awal *linked list* dapat dilakukan dengan mengatur hubungan pointer antar node.

Fungsi `insertEnd_2027()` digunakan untuk menambahkan node di bagian akhir *linked list*. Program akan membuat node baru terlebih dahulu, kemudian memeriksa apakah *linked list* masih kosong atau tidak. Jika kosong, maka node baru langsung dijadikan *head*. Namun jika *linked list* sudah memiliki data, program akan melakukan traversal dari *head* hingga mencapai node terakhir dengan menggunakan perulangan `while`. Setelah node terakhir ditemukan, pointer `next_2027` dari node terakhir dihubungkan ke node baru dan pointer `prev_2027` dari node baru diarahkan kembali ke node sebelumnya. Dengan cara tersebut, hubungan dua arah antar node tetap terjaga dengan baik.

Fungsi `insertAtPosition_2027()` digunakan untuk menambahkan node pada posisi tertentu di dalam *linked list*. Program pertama kali membuat node baru, kemudian memeriksa apakah posisi yang diminta adalah posisi pertama. Jika posisi pertama, maka proses penambahan dilakukan seperti pada fungsi *insert* di awal. Jika bukan, program melakukan traversal menggunakan perulangan `for` hingga mencapai posisi sebelum lokasi penyisipan. Setelah posisi ditemukan, pointer

prev\_2027 dan next\_2027 dari node baru diatur agar terhubung dengan node sebelumnya dan node sesudahnya. Program juga memperbarui pointer node lain yang terkait agar struktur *linked list* tetap tersusun dengan benar. Jika posisi yang dimasukkan tidak tersedia, maka program akan menampilkan pesan “Posisi tidak ada”.

Fungsi printList\_2027() digunakan untuk menampilkan isi *double linked list*. Fungsi ini melakukan traversal mulai dari *head* hingga node terakhir menggunakan pointer next\_2027. Setiap data node akan dicetak ke layar dengan tanda <-> sebagai simbol hubungan antar node pada *double linked list*. Fungsi ini membantu pengguna melihat hasil perubahan *linked list* setelah dilakukan operasi penambahan data.

Pada fungsi main, program membuat *linked list* awal dengan data 2 <- 3 <-> 5. Setelah itu, program melakukan beberapa operasi penambahan node, yaitu menambahkan node bernilai 1 di awal *linked list*, menambahkan node berniali 6 di akhir *linked list*, dan menambahkan node bernilai 4 pada posisi keempat. Setelah setiap operasi dilakukan, isi *linked list* akan ditampilkan menggunakan fungsi printList\_2027().

## 2.4 Class PenelusuranDLL

Berikut kode program Java mengenai class PenelusuranDLL :

```
public class PenelusuranDLL_2511532027 {
    // fungsi penelusuran maju
    static void forwardTraversal_2027(NodeDLL_2511532027 head_2027) {
        // memulai penelusuran dari head
        NodeDLL_2511532027 curr_2027 = head_2027;
        // lanjutkan sampai akhir
        while (curr_2027 != null) {
            // print data
            System.out.print(curr_2027.data_2027 + " <-> ");
            // pindah ke node berikutnya
            curr_2027 = curr_2027.next_2027;
        }
        // print spasi
        System.out.println();
    }
    // fungsi penelusuran mundur
```

```

static void backwardTraversal_2027(NodeDLL_2511532027 tail_2027)
{
    // mulai dari akhir
    NodeDLL_2511532027 curr_2027 = tail_2027;
    // lanjut sampai head
    while (curr_2027 != null) {
        // cetak data
        System.out.print(curr_2027.data_2027 + " <-> ");
        // pindah ke node sebelumnya
        curr_2027 = curr_2027.prev_2027;
    }
    // cetak spasi
    System.out.println();
}

public static void main (String[] args) {
    // cetak DLL
    NodeDLL_2511532027 head_2027 = new NodeDLL_2511532027(1);
    NodeDLL_2511532027 second_2027 = new NodeDLL_2511532027(2);
    NodeDLL_2511532027 third_2027 = new NodeDLL_2511532027(3);

    head_2027.next_2027 = second_2027;
    second_2027.prev_2027 = head_2027;
    second_2027.next_2027 = third_2027;
    third_2027.prev_2027 = second_2027;

    System.out.println("Penelusuran maju: ");
    forwardTraversal_2027(head_2027);

    System.out.println("Penelusuran mundur: ");
    backwardTraversal_2027(third_2027);
}
}

```

Kode program PenelusuranDLL\_2511532027 digunakan untuk melakukan proses traversal atau penelusuran pada struktur data *double linked list*. Program ini menunjukkan dua jenis traversal yang dapat dilakukan pada *double linked list*, yaitu penelusuran maju dan penelusuran mundur. Keunggulan utama *double linked list*

dibandingkan *single linked list* terlihat pada kemampuan traversal dua arah karena setiap node memiliki pointer menuju node berikutnya dan node sebelumnya.

Fungsi `forwardTraversal_2027()` digunakan untuk melakukan penelusuran data dari depan ke belakang. Fungsi ini menerima parameter berupa `head_2027` yang merupakan node pertama dalam *linked list*. Program kemudian membuat variabel `curr_2027` sebagai penunjuk node yang sedang diproses dan menginisialisasinya dengan `head`. Selanjutnya, perulangan `while` digunakan untuk menelusuri seluruh node selama nilai `curr_2027` tidak bernilai `null`. Pada setiap perulangan, data pada node dicetak ke layar menggunakan `System.out.print`, kemudian pointer dipindahkan ke node berikutnya melalui `curr_2027 = curr_2027.next_2027`. Program ini terus berlangsung hingga mencapai node terakhir. Fungsi ini menunjukkan cara traversal maju pada *double linked list* menggunakan pointer `next`.

Fungsi `backwardTraversal_2027()` digunakan untuk melakukan penelusuran dari belakang ke depan. Fungsi ini menerima parameter `tail_2027` yang merupakan node terakhir dalam *linked list*. Sama seperti traversal maju, program menggunakan variabel `curr_2027` sebagai penunjuk node yang sedang diproses. Perbedaannya terletak pada arah perpindahan node, yaitu menggunakan pointer `prev_2027`. Pada setiap perulangan, data node dicetak ke layar, kemudian pointer dipindahkan ke node sebelumnya menggunakan `curr_2027 = curr_2027.prev_2027`. Proses ini dilakukan hingga mencapai `head` atau hingga nilai pointer menjadi `null`. Fungsi ini membuktikan bahwa *double linked list* dapat ditelusuri dari arah belakang tanpa perlu kembali ke `head` terlebih dahulu.

Pada fungsi main, program membuat sebuah *double linked list* sederhana yang terdiri dari tiga node dengan data 1, 2, dan 3. Setiap node saling dihubungkan menggunakan pointer `next_2027` dan `prev_2027` sehingga membentuk hubungan dua arah. Setelah *linked list* selesai dibuat, program memanggil fungsi `forwardTraversal_2027()` untuk menampilkan data dari depan ke belakang dan fungsi `backwardTraversal_2027()` untuk menampilkan data dari belakang ke depan.

## 2.5 Class HapusDLL

Berikut kode program Java mengenai class `HapusDLL` :

```
public class HapusDLL_2511532027 {  
    // fungsi menghapus node awal
```

```

        public static NodeDLL_2511532027 delHead_2027(NodeDLL_2511532027
head_2027) {
            if (head_2027 == null) {
                return null; }

            NodeDLL_2511532027 temp_2027 = head_2027;
            head_2027 = head_2027.next_2027;
            if (head_2027 != null) {
                head_2027.prev_2027 = null; }
            return head_2027;
        }
        // fungsi menghapus di akhir
        public static NodeDLL_2511532027 dellast_2027(NodeDLL_2511532027
head_2027) {
            if (head_2027 == null) {
                return null; }

            if (head_2027.next_2027 == null) {
                return null; }

            NodeDLL_2511532027 curr_2027 = head_2027;
            while (curr_2027.next_2027 != null) {
                curr_2027 = curr_2027.next_2027;
            }
            // update pointer previous node
            if (curr_2027.prev_2027 != null) {
                curr_2027.prev_2027.next_2027 = null; }
            return head_2027;
        }
        // fungsi menghapus node di posisi tertentu
        public static NodeDLL_2511532027 delPos_2027(NodeDLL_2511532027
head_2027, int pos_2027) {
            // jika DLL kosong
            if (head_2027 == null) {
                return head_2027; }

            NodeDLL_2511532027 curr_2027 = head_2027;
            // telusuri sampai ke node yang akan dihapus
            for (int i_2027 = 1; curr_2027 != null && i_2027 <
pos_2027; ++i_2027) {
                curr_2027 = curr_2027.next_2027; }
            // jika posisi tidak ditemukan

```

```

        if (curr_2027 == null) {
            return head_2027; }
        // update pointer
        if (curr_2027.prev_2027 != null) {
            curr_2027.prev_2027.next_2027 = curr_2027.next_2027;
        }

        if (curr_2027.next_2027 != null) {
            curr_2027.next_2027.prev_2027 = curr_2027.prev_2027;
        }

        // jika yang dihapus head
        if (head_2027 == curr_2027) {
            head_2027 = curr_2027.next_2027; }
        return head_2027;
    }

    // fungsi mencetak DLL
    public static void printList_2027 (NodeDLL_2511532027 head_2027)
    {
        NodeDLL_2511532027 curr_2027 = head_2027;
        while (curr_2027 != null) {
            System.out.print(curr_2027.data_2027 + " <-> ");
            curr_2027 = curr_2027.next_2027;
        }
        System.out.println();
    }

    public static void main (String[] args) {
        // buat sebuah DLL
        NodeDLL_2511532027 head_2027 = new NodeDLL_2511532027(1);
        head_2027.next_2027 = new NodeDLL_2511532027(2);
        head_2027.next_2027.prev_2027 = head_2027;
        head_2027.next_2027.next_2027 = new NodeDLL_2511532027(3);
        head_2027.next_2027.next_2027.prev_2027 =
head_2027.next_2027;
        head_2027.next_2027.next_2027.next_2027 = new
NodeDLL_2511532027(4);
        head_2027.next_2027.next_2027.next_2027.prev_2027 =
head_2027.next_2027.next_2027;
        head_2027.next_2027.next_2027.next_2027.next_2027 = new
NodeDLL_2511532027(5);
    }
}

```

```

        head_2027.next_2027.next_2027.next_2027.next_2027.prev_2027
= head_2027.next_2027.next_2027.next_2027;

        System.out.print("DLL Awal: ");
        printList_2027(head_2027);

        System.out.print("Setelah head dihapus: ");
        head_2027 = delHead_2027(head_2027);
        printList_2027(head_2027);

        System.out.print("Setelah node terakhir dihapus: ");
        head_2027 = delLast_2027(head_2027);
        printList_2027(head_2027);

        System.out.print("Menghapus node ke 2: ");
        head_2027 = delPos_2027(head_2027, 2);

        printList_2027(head_2027);
    }
}

```

Kode program HapusDLL\_2511532027 digunakan untuk melakukan operasi penghapusan node pada struktur data *double linked list*. Program ini menyediakan beberapa fungsi penghapusan, yaitu menghapus node di awal *linked list*, menghapus node di akhir *linked list*, dan menghapus node pada posisi tertentu. Selain itu, program juga memiliki fungsi untuk mencetak isi *linked list* sehingga perubahan data setelah proses penghapusan dapat dilihat dengan jelas. Operasi *deletion* pada *double linked list* dilakukan dengan memperbarui pointer *next* dan *prev* agar hubungan antar node tetap terhubung dengan benar setelah suatu node dihapus.

Fungsi `delHead_2027()` digunakan untuk menghapus node pertama atau *head* dari *double linked list*. Program pertama kali memeriksa apakah *linked list* kosong dengan kondisi `head_2027 == null`. Jika kosong, fungsi akan mengembalikan nilai `null`. Namun jika *linked list* memiliki data, maka *head* lama disimpan sementara ke dalam variabel `temp_2027`, kemudian *head* dipindahkan ke node berikutnya menggunakan `head_2027 = head_2027.next_2027`. Setelah itu,



pointer `prev_2027` pada *head* baru diubah menjadi null karena node tersebut sekarang menjadi node pertama dan tidak memiliki node sebelumnya. Fungsi ini menunjukkan proses penghapusan node awal dengan memperbarui posisi *head* dan hubungan pointer antar node.

Fungsi `delLast_2027()` digunakan untuk menghapus node terakhir pada *linked list*. Program terlebih dahulu memeriksa apakah *linked list* kosong atau hanya memiliki satu node. Jika hanya terdapat satu node, maka *linked list* akan menjadi kosong setelah penghapusan sehingga fungsi mengembalikan null. jika *linked list* memiliki lebih dari satu node, program melakukan traversal menggunakan perulangan `while` hingga mencapai node terakhir. Setelah node terakhir ditemukan, pointer `next_2027` pada node sebelumnya diubah menjadi null sehingga hubungan dengan node terakhir terputus. Dengan cara ini, node terakhir berhasil dihapus dari *linked list* tanpa merusak hubungan node lainnya.

Fungsi `delPos_2027()` digunakan untuk menghapus node pada posisi tertentu. Program pertama kali memeriksa apakah *linked list* kosong. Jika tidak kosong, program melakukan traversal menggunakan perulangan `for` hingga mencapai posisi node yang akan dihapus. Setelah node ditemukan, pointer dari node sebelumnya dan node sesudahnya diperbarui agar saling terhubung langsung. Pointer `next_2027` dari node sebelumnya diarahkan ke node sesudahnya, sedangkan pointer `prev-2027` dari node sesudahnya diarahkan ke node sebelumnya. Jika node yang dihapus merupakan *head*, maka posisi *head* dipindahkan ke node berikutnya. Fungsi ini menunjukkan bahwa proses penghapusan pada *double linked list* memerlukan pengaturan dua arah pointer agar struktur list tetap valid.

Fungsi `printList_2027()` digunakan untuk menampilkan isi *linked list* dari depan ke belakang. Program melakukan traversal mulai dari *head* hingga node terakhir menggunakan pointer `next_2027`. Setiap node akan dicetak ke layar dengan simbol `<->` untuk menunjukkan hubungan antar node pada *double linked list*. Fungsi ini membantu pengguna melihat hasil perubahan *linked list* setelah dilakukan operasi penghapusan.

Pada fungsi `main`, program membuat sebuah *double linked list* dengan data 1, 2, 3, 4, 5. Setelah *linked list* terbentuk, program menampilkan isi awal *linked list* menggunakan fungsi `printList-2027()`. Selanjutnya, program

melakukan beberapa operasi penghapusan, yaitu `delhead-2027()`, `delLast_2027()`, dan `delPos_2027()`. Setelah setiap proses penghapusan selesai, isi *linked list* kembali ditampilkan sehingga perubahan struktur data dapat diamati dengan jelas.

## 2.6 Tugas Praktikum Pekan 6

Berikut tugas praktikum yang diberikan :

### Tugas Praktikum Struktur Data Pekan 6

**Topik: Double Linked List**

**Tema: Playlist Lagu Sederhana**

#### 1. Deskripsi Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (`prev`) dan lagu berikutnya (`next`). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

#### 2. Aturan Penamaan (Wajib)

Ikuti aturan yang sama seperti praktikum sebelumnya:

- Nama Kelas: pakai NIM lengkap. Contoh: `Lagu_2411531005` dan `Musik_2411531005`
- Nama variabel & method: pakai 4 digit terakhir NIM. Contoh: `judul_1005`, `tambahLagu_1005()`
- Pointer: `next_1005` dan `prev_1005` juga wajib pakai akhiran NIM

#### 3. Struktur yang Dibuat

##### Kelas Lagu\_NIM

Atribut yang harus ada:

- `judul` (String)
- `penyanyi` (String)
- `next` (pointer ke Lagu)
- `prev` (pointer ke Lagu)

Method:

- constructor
- getter
- setter

##### Kelas Musik\_NIM

- Buat 5 operasi utama saja (nama method diakhiri 4 digit NIM):

1. `tambahLagu_xxxx()` : menambah lagu baru di AKHIR playlist
2. `hapusLaguAwal_xxxx()` : menghapus lagu pertama (head)

Gambar 2.1

3. tampilMaju\_xxxx() : menampilkan semua lagu dari awal ke akhir
4. tampilMundur\_xxxx() : menampilkan semua lagu dari akhir ke awal (wajib DLL)
5. cariLagu\_xxxx(judul) : mencari lagu berdasarkan judul (tidak case-sensitive)

#### 4. Contoh Tampilan Menu

== Playlist Musik NIM: 2411531005 ==

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1

Judul: Rosalia - Mio Cristo Piange Diamanti

Penyanyi: Rosalia

Lagu berhasil ditambahkan!

#### 5. Yang Dinilai

1. Pointer next dan prev terhubung dengan benar
2. Aturan penamaan NIM dipatuhi 100%
3. Program tidak error saat playlist kosong
4. Fitur tampil mundur berfungsi
5. Kode rapi dan ada komentar singkat

Gambar 2.2

Berikut kode program Java dari tugas praktikum :

```
public class Lagu_2511532027 {

    // atribut lagu
    String judul_2027;
    String penyanyi_2027;

    // pointer DLL
    Lagu_2511532027 next_2027;
    Lagu_2511532027 prev_2027;

    // konstruktor
    public Lagu_2511532027(String judul_2027, String penyanyi_2027) {
        this.judul_2027 = judul_2027;
        this.penyanyi_2027 = penyanyi_2027;
        this.next_2027 = null;
        this.prev_2027 = null;
    }
}
```

```

// getter judul
public String getJudul_2027() {
    return judul_2027;
}

// getter penyanyi
public String getPenyanyi_2027() {
    return penyanyi_2027;
}

// setter judul
public void setJudul_2027(String judul_2027) {
    this.judul_2027 = judul_2027;
}

// setter penyanyi
public void setPenyanyi_2027(String penyanyi_2027) {
    this.penyanyi_2027 = penyanyi_2027;
}
}

```

Kode program Lagu\_2511532027 digunakan untuk mendefinisikan sebuah node pada struktur data *double linked list* yang digunakan dalam program playlist musik. Kelas ini berfungsi sebagai tempat penyimpanan data lagu sekaligus penghubung antar node di dalam *linked list*. Setiap objek yang dibuat dari kelas lagu\_2511532027 merepresentasikan satu buah lagu yang memiliki informasi judul lagu, nama penyanyi, serta pointer yang menghubungkan node ke node lainnya.

Pada bagian atribut, terdapat variabel judul-2027 dan penyanyi\_2027 yang bertipe String. Variabel judul\_2027 digunakan untuk menyimpan nama atau judul lagu, sedangkan penyanyi\_2027 digunakan untuk menyimpan nama penyanyi dari lagu tersebut. Selain atribut data, kelas ini juga memiliki dua buah pointer yaitu next\_2027 dan prev-2027. Pointer next\_2027 digunakan untuk menunjuk node berikutnya, sedangkan pointer prev\_2027 digunakan untuk menunjuk node sebelumnya. Keberadaan dua pointer tersebut merupakan ciri utama dari *double*

*linked list* karena memungkinkan traversal dilakukan dari depan ke belakang maupun dari belakang ke depan.

Program juga memiliki konstruktor `Lagu_2511532027(String judul_2027, String penyanyi_2027)` yang digunakan untuk menginisialisasi objek lagu saat pertama kali dibuat. Konstruktor menerima parameter berupa judul lagu dan nama penyanyi, kemudian menyimpannya ke dalam atribut menggunakan *keyword* `this`. Selain itu, pointer `next_2027` dan `prev_2027` diatur bernilai `null` karena node yang baru dibuat sbelum terhubung dengan node lain pada *linked list*. Konstruktor ini mempermudah proses pembuatan node baru ketika pengguna menambahkan lagu ke dalam playlist.

Selain konstruktor, kelas ini juga memiliki method getter dan setter. Method getter terdiri dari `getJudul_2027()` dan `getPenyanyi_2027()` yang digunakan untuk mengambil nilai atribut judul dan penyanyi dari suatu objek lagu. Sementara itu, method setter terdiri dari `setJudul_2027()` dan `setPenyanyi_2027()` yang digunakan untuk mengubah nilai atribut judul maupun penyanyi. Penggunaan getter dan setter membantu menerapkan konsep enkapsulasi dalam pemrograman berorientasi objek sehingga data dapat diakses dan diubah melalui method yang telah disediakan.

```
import java.util.Scanner;

public class Musik_2511532027 {

    // head dan tail
    Lagu_2511532027 head_2027 = null;
    Lagu_2511532027 tail_2027 = null;

    // tambah lagu di akhir
    public void tambahLagu_2027(String judul_2027, String penyanyi_2027)
    {

        Lagu_2511532027 laguBaru_2027 = new Lagu_2511532027(judul_2027,
        penyanyi_2027);

        // jika playlist kosong
        if (head_2027 == null) {
            head_2027 = laguBaru_2027;
```

```

tail_2027 = laguBaru_2027;
    } else {
        tail_2027.next_2027 = laguBaru_2027;
        laguBaru_2027.prev_2027 = tail_2027;
        tail_2027 = laguBaru_2027;
    }
    System.out.println("Lagu berhasil ditambahkan!");
}

// hapus lagu awal
public void hapusLaguAwal_2027() {

    // jika kosong
    if (head_2027 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    System.out.println("Lagu \"" + head_2027.judul_2027 + "\"
berhasil dihapus.");

    // jika hanya 1 lagu
    if (head_2027 == tail_2027) {
        head_2027 = null;
        tail_2027 = null;
    } else {
        head_2027 = head_2027.next_2027;
        head_2027.prev_2027 = null;
    }
}

// tampil maju
public void tampilMaju_2027() {

    if (head_2027 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532027 bantu_2027 = head_2027;

```

```

        System.out.println("=== Playlist Maju ===");
        while (bantu_2027 != null) {
            System.out.println(bantu_2027.judul_2027 + " - " +
bantu_2027.penyanyi_2027);
            bantu_2027 = bantu_2027.next_2027;
        }
    }

    // tampil mundur
    public void tampilMundur_2027() {
        if (tail_2027 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532027 bantu_2027 = tail_2027;
        System.out.println("=== Playlist Mundur ===");
        while (bantu_2027 != null) {
            System.out.println(bantu_2027.judul_2027 + " - " +
bantu_2027.penyanyi_2027);
            bantu_2027 = bantu_2027.prev_2027;
        }
    }

    // cari lagu
    public void cariLagu_2027(String judul_2027) {
        if (head_2027 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532027 bantu_2027 = head_2027;
        boolean ketemu_2027 = false;
        while (bantu_2027 != null) {
            if (bantu_2027.judul_2027.equalsIgnoreCase(judul_2027)) {
                System.out.println("Lagu ditemukan!");
                System.out.println("Judul : " + bantu_2027.judul_2027);
                System.out.println("Penyanyi : " +
bantu_2027.penyanyi_2027);
                ketemu_2027 = true;
            }
            bantu_2027 = bantu_2027.next_2027;
        }
    }
}

```

```

        break;
    }
    bantu_2027 = bantu_2027.next_2027;
}
if (!ketemu_2027) {
    System.out.println("Lagu tidak ditemukan!");
}
}

// main program
public static void main(String[] args) {
    Scanner input_2027 = new Scanner(System.in);
    Musik_2511532027 musik_2027 = new Musik_2511532027();
    int pilihan_2027;
    do {
        System.out.println("\n=== Playlist Musik NIM: 2511532027
===");

        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");

        System.out.print("Pilihan: ");
        pilihan_2027 = input_2027.nextInt();
        input_2027.nextLine();

        switch (pilihan_2027) {
            case 1:
                System.out.print("Judul Lagu: ");
                String judul_2027 = input_2027.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi_2027 = input_2027.nextLine();
                musik_2027.tambahLagu_2027( judul_2027,
                penyanyi_2027);
                break;

```



```

        case 2:
            musik_2027.hapusLaguAwal_2027();
            break;

        case 3:
            musik_2027.tampilMaju_2027();
            break;

        case 4:
            musik_2027.tampilMundur_2027();
            break;

        case 5:
            System.out.print("Masukkan judul lagu: ");
            String cari_2027 = input_2027.nextLine();
            musik_2027.cariLagu_2027(cari_2027);
            break;

        case 6:
            System.out.println("Program selesai.");
            break;
        default:
            System.out.println("Pilihan tidak valid!");
    }
} while (pilihan_2027 != 6);
input_2027.close();
}
}

```

Kode program Musik\_2511532027 digunakan untuk mengelola playlist lagu menggunakan struktur data *double linked list*. Kelas ini berfungsi sebagai pengatur seluruh operasi yang dapat dilakukan pada playlist, seperti menambahkan lagu, menghapus lagu, menampilkan daftar lagu, dan mencari lagu berdasarkan judul. Pada program ini, setiap lagu direpresentasikan sebagai node dari kelas Lagu\_2511532027 yang saling terhubung menggunakan pointer *next* dan *prev*. Dengan menggunakan *double linked list*, playlist dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Pada bagian awal kelas terdapat dua variabel, yaitu `head-2027` dan `tail-2027`. Variabel `head_2027` digunakan untuk menunjuk node pertama pada playlist, sedangkan `tail-2027` digunakan untuk menunjuk node terakhir. Kedua pointer ini sangat penting dalam *double linked list* karena mempermudah proses penambahan data di akhir list secara traversal mundur dari node terakhir menuju node pertama. Pada kondisi awal, kedua pointer bernilai *null* karena playlist masih kosong.

Method `tambahlagu_2027()` digunakan untuk menambahkan lagu baru ke bagian akhir playlist. Program terlebih dahulu membuat objek baru dari kelas `Lagu_2511532027` yang berisi judul dan nama penyanyi. Jika playlist masih kosong, maka node baru akan dijadikan sebagai `head_2027` dan `tail_2027`. namun jika playlist sudah memiliki data, pointer `next_2027` pada node terakhir akan diarahkan ke node baru, sedangkan pointer `prev-2027` pada node baru diarahkan ke node terakhir sebelumnya. Setelah itu, posisi `tail-2027` dipindahkan ke node baru. Method ini menunjukkan cara penambahan node pada *double linked list* dengan tetap menjaga hubungan dua arah antar node.

Method `hapusLaguAwal_2027()` digunakan untuk menghapus lagu pertama pada playlist. Program terlebih dahulu memeriksa apakah playlist kosong. Jika kosong, program akan menampilkan pesan bahwa playlist tidak memiliki data. Jika playlist hanya memiliki satu lagu, maka `head_2027` dan `tail_2027` diubah menjadi *null*. Namun jika terdapat lebih dari satu lagu, posisi `head_2027` dipindahkan ke node berikutnya dan pointer `prev_2027` pada *head* baru diatur menjadi *null*. Dengan cara ini, node pertama berhasil dihapus tanpa merusak hubungan antar node lainnya.

Method `tampilMaju_2027()` digunakan untuk menampilkan seluruh lagu dari awal ke akhir playlist. Program melakukan traversal mulai dari `head_2027` menggunakan pointer `next_2027` hingga mencapai node terakhir. Setiap node yang dilewati akan dicetak ke layar berupa judul lagu dan nama penyanyi. Sebaliknya, method `tampilMundur_2027()` digunakan untuk menampilkan playlist dari belakang ke depan menggunakan pointer `prev_2027` dimulai dari `tail_2027`. kedua method ini menunjukkan kelebihan *double linked list* yang dapat melakukan traversal dalam dua arah.

Method cariLagu\_2027() digunakan untuk mencari lagu berdasarkan judul lagu. Program melakukan traversal dari head\_2027 hingga node terakhir sambil membandingkan judul lagu menggunakan method equalsIgnoreCase(). Method ini memungkinkan pencarian dilakukan tanpa membedakan huru besar dan huruf kecil. Jika lagu ditemukan, program akan menampilkan informasi lagu berupa judul dan nama penyanyi. Namun jika data tidak ditemukan, program akan menampilkan pesan bahwa lagu tidak ditemukan.

Pada method main, program menggunakan Scanner untuk menerima input dari pengguna. Program kemudian menampilkan menu pilihan yang terdiri dari tambah lagu, hapus lagu pertama, tampil playlist maju, tampil playlist mundur, cari lagu, dan keluar dari program. Penggunaan struktur perulangan do-while membuat menu terus muncul hingga pengguna memilih keluar. Seluruh operasi pada menu dijalankan menggunakan switch-case sehingga program dapat memproses pilihan pengguna dengan lebih teratur.

Hasil / Output :

```
=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Playlist kosong!

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist kosong!

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Playlist kosong!

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: lavie
Playlist kosong!
```

Gambar 2.3

```

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: lavie
Penyanyi: n25
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: phony
Penyanyi: n25
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: parasol cider
Penyanyi: mmj
Lagu berhasil ditambahkan!

```

Gambar 2.4

```

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
lavie - n25
phony - n25
parasol cider - mmj

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Playlist Mundur ===
parasol cider - mmj
phony - n25
lavie - n25

```

Gambar 2.5

```

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: lavie
Lagu ditemukan!
Judul : lavie
Penyanyi : n25

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "lavie" berhasil dihapus.

```

Gambar 2.6

```

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
phony - n25
parasol cider - mmj

=== Playlist Musik NIM: 2511532027 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Playlist Mundur ===
parasol cider - mmj
phony - n25

```

Gambar 2.7

### BAB 3

#### KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Double Linked List* (DLL) merupakan struktur data linear yang memiliki hubungan dua arah antar node melalui pointer *next* dan *prev*. *Double linked list* mempermudah proses pengelolaan data seperti penambahan, penghapusan, pencarian, dan penelusuran data baik dari depan maupun dari belakang. Melalui implementasi program yang telah dibuat, pemahaman mengenai cara kerja node, hubungan antar pointer, serta manipulasi data pada *double linked list* juga menunjukkan bahwa struktur data ini lebih fleksibel dibandingkan *single linked list* dalam beberapa operasi tertentu karena setiap node dapat langsung mengakses node sebelumnya maupun node berikutnya.

Dalam pembuatan program *double linked list*, diperlukan ketelitian dalam mengatur pointer *next* dan *prev* agar hubungan antar node tetap benar dan tidak menyebabkan error pada program. Oleh karena itu, disarankan untuk lebih memahami alur perpindahan pointer setiap operasi seperti *insertion*, *deletion*, dan *traversal* sebelum melakukan implementasi program. Praktik dan latihan yang lebih banyak juga diperlukan agar kemampuan memahami struktur data dan pemrograman menjadi semakin baik.

## DAFTAR PUSTAKA

- [1] <https://www.geeksforgeeks.org/dsa/doubly-linked-list/> [Diakses 12 Mei 2026]
- [2] <https://www.programiz.com/dsa/doubly-linked-list> [Diakses 12 Mei 2026]
- [3] <https://socs.binus.ac.id/2017/03/15/doubly-linked-list/> [Diakses 12 Mei 2026]
- [4] <https://opensa-server.cs.vt.edu/ODSA/Books/CS2/html/ListDouble.html>  
[Diakses 12 Mei 2026]

## **LAMPIRAN**

|                  |    |
|------------------|----|
| Gambar 2.1 ..... | 15 |
| Gambar 2.2 ..... | 16 |
| Gambar 2.3 ..... | 24 |
| Gambar 2.4 ..... | 25 |
| Gambar 2.5 ..... | 25 |
| Gambar 2.6 ..... | 26 |
| Gambar 2.7 ..... | 26 |